Module 4 — JavaScript

> **Objectif** : apprendre à programmer en JavaScript, manipuler dynamiquement le contenu d'une page (DOM), réagir aux interactions utilisateur (événements) et communiquer avec un serveur (`fetch`).

---

1. Qu'est-ce que JavaScript ?

JavaScript (ou **JS**) est le **langage de programmation du Web**. Créé en 1995 par Brendan Eich (Netscape) en 10 jours, c'est aujourd'hui l'un des langages les plus utilisés au monde.

	JavaScript
Type	Langage de programmation interprété, dynamique
Standard	ECMAScript (ES) — version annuelle (ES2025 actuelle)
Extensions	`.js` , `.mjs` , `.cjs`
S'exécute dans	Navigateurs (front), Node.js / Deno / Bun (back), Electron (desktop), React Native (mobile)

⚠️ **JavaScript ≠ Java**. Aucun rapport (sauf le nom, choix marketing en 1995).

Pourquoi JavaScript ?

- **Universel** : tourne dans tous les navigateurs sans installation.
- **Polyvalent** : front-end, back-end, mobile, desktop, IoT.
- **Énorme écosystème** : npm (plus de 2 millions de packages).
- **Demande de marché** : c'est le langage le plus recherché en développement web.

---

2. Intégrer JavaScript à une page HTML

2.1 JS externe (recommandé)

`script.js` :

```
console.log("Bonjour le monde !");
```

Dans `index.html` :

```
<script src="script.js" defer></script>
```

Attribut	Effet
(rien)	Téléchargement et exécution **bloquent** le parsing HTML
`defer`	Téléchargement en parallèle, exécution **après** le parsing
`async`	Téléchargement en parallèle, exécution **dès que prêt** (ordre non garanti)

💡 Utilisez `defer` la plupart du temps, et placez les `<script>` dans le `<head>`.

2.2 JS interne

```
<script>
  console.log("Inline");
</script>
```

2.3 La console

```
console.log("Message normal");
console.warn("Attention !");
console.error("Erreur !");
console.table([{ nom: "Alice" }, { nom: "Bob" }]);
console.time("calcul"); /* … */ console.timeEnd("calcul");
```

Ouvrez **F12** → **Console** pour les voir.

---

3. Variables

Déclarer une variable

```
let age = 25;              // Variable modifiable
const PI = 3.14159;        // Constante (non réassignable)
var ancienneFacon = "obsolète"; // À éviter en JS moderne
```

	`let`	`const`	`var`
Réassignable	✅	❌	✅
Portée	Bloc `{ }`	Bloc `{ }`	Fonction
Hoisting	Non utilisable avant déclaration	Idem	Utilisable (mais `undefined`)

💡 **Règle moderne** : utilisez `const` par défaut, `let` si la valeur change, **jamais** `var`.

Règles de nommage

- Commence par une lettre, `_` ou `$`.
- Sensible à la casse (`age` ≠ `Age`).
- Convention **camelCase** : `monAge`, `nombreDePages`, `estConnecte`.
- Constantes globales en **UPPER_SNAKE_CASE** : `MAX_TENTATIVES`.

---

4. Types de données

JavaScript est **dynamiquement typé** : on ne déclare pas le type, il est déduit.

Types primitifs

Type	Exemple
`string`	`"Bonjour"`, `'Hello'`, `` `template ${x}` ``
`number`	`42`, `3.14`, `-0.5`, `Infinity`, `NaN`
`boolean`	`true`, `false`
`null`	`null` (absence intentionnelle de valeur)
`undefined`	`undefined` (variable non initialisée)
`symbol`	`Symbol("id")` (avancé)
`bigint`	`9007199254740992n` (grands entiers)

```
typeof "Bonjour"   // "string"
typeof 42          // "number"
typeof true        // "boolean"
typeof null        // "object" (bug historique)
typeof undefined   // "undefined"
typeof [1, 2]      // "object"
```

Chaînes (strings)

```
const prenom = "Kokou";
const nom = 'AGBAVON';

// Concaténation
const complet = prenom + " " + nom;

// Template literals (recommandé)
const message = `Bonjour ${prenom}, vous avez ${10 + 5} messages.`;

// Multi-lignes
const html = `
  <article>
    <h2>${prenom}</h2>
  </article>
`;

// Méthodes utiles
"bonjour".length          // 7
"bonjour".toUpperCase()   // "BONJOUR"
"bonjour".includes("jour") // true
"bonjour".slice(0, 3)     // "bon"
"  hi  ".trim()           // "hi"
"a,b,c".split(",")        // ["a", "b", "c"]
"abc".repeat(3)           // "abcabcabc"
"Hello".replace("l", "L") // "HeLlo" (remplace la 1ère occurrence)
```

Nombres

```
const a = 10;
const b = 3;

a + b   // 13
a - b   // 7
a * b   // 30
a / b   // 3.3333
a % b   // 1   (modulo : reste de la division)
a ** b  // 1000 (puissance)

Math.PI          // 3.14159…
Math.round(4.6)  // 5
Math.floor(4.9)  // 4
Math.ceil(4.1)   // 5
Math.abs(-7)     // 7
Math.min(3, 5)   // 3
Math.max(3, 5)   // 5
Math.random()    // Nombre aléatoire entre 0 et 1
Math.sqrt(16)    // 4

// Conversion
Number("42")     // 42
parseInt("42px") // 42
parseFloat("3.14") // 3.14
String(42)       // "42"
(42).toFixed(2)  // "42.00"
```

---

5. Opérateurs

Arithmétiques

`+`, `-`, `*`, `/`, `%`, `**`

```
let x = 5;
x++;       // x = 6
x--;       // x = 5
x += 3;    // x = 8 (équivalent à x = x + 3)
x -= 2;    // x = 6
x *= 2;    // x = 12
```

Comparaison

Opérateur	Sens
`==`	Égalité avec conversion de type (à éviter)
`===`	**Égalité stricte** (sans conversion)
`!=`	Différent (avec conversion)
`!==`	Différent strict
`<`, `>`, `<=`, `>=`	Comparaison numérique

```
5 == "5"    // true (conversion)
5 === "5"   // false (types différents)
0 == false  // true
0 === false // false
```

⚠️ **Toujours** utiliser `===` et `!==`.

Logiques

Opérateur	Sens
`&&`	ET
`||`	OU
`!`	NON
`??`	Coalescence des nuls (utilise la valeur de droite si gauche est `null` ou `undefined`)

```
true && false   // false
true || false   // true
!true           // false

const nom = utilisateur.nom ?? "Anonyme";
```

---

6. Conditions

`if / else if / else`

```
const age = 18;

if (age >= 18) {
  console.log("Majeur");
} else if (age >= 13) {
  console.log("Adolescent");
} else {
  console.log("Enfant");
}
```

Opérateur ternaire (condition courte)

```
const statut = age >= 18 ? "Majeur" : "Mineur";
```

`switch`

```
const jour = "lundi";

switch (jour) {
  case "lundi":
  case "mardi":
    console.log("Début de semaine");
    break;
  case "samedi":
  case "dimanche":
    console.log("Week-end");
    break;
  default:
    console.log("Milieu de semaine");
}
```

---

7. Boucles

`for` classique

```
for (let i = 0; i < 5; i++) {
  console.log(i);   // 0, 1, 2, 3, 4
}
```

`while`

```
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

`do … while`

```
let i = 0;
do {
  console.log(i);
  i++;
} while (i < 5);
```

`for … of` (parcourir un tableau)

```
const fruits = ["pomme", "banane", "mangue"];

for (const fruit of fruits) {
  console.log(fruit);
}
```

`for … in` (parcourir les clés d'un objet)

```
const personne = { nom: "Alice", age: 25 };

for (const cle in personne) {
  console.log(cle, personne[cle]);
}
```

Contrôle de boucle

- `break` — sort de la boucle

- `continue` — passe à l'itération suivante

```
for (let i = 0; i < 10; i++) {
  if (i === 3) continue;
  if (i === 7) break;
  console.log(i);  // 0, 1, 2, 4, 5, 6
}
```

---

8. Fonctions

Déclaration classique

```
function saluer(nom) {
  return `Bonjour ${nom} !`;
}

console.log(saluer("Alice"));  // "Bonjour Alice !"
```

Expression de fonction

```
const saluer = function(nom) {
  return `Bonjour ${nom} !`;
};
```

Fonction fléchée (arrow function)

```
const saluer = (nom) => `Bonjour ${nom} !`;

// Avec corps
const additionner = (a, b) => {
  const somme = a + b;
  return somme;
};

// Un seul paramètre : parenthèses optionnelles
const carre = x => x * x;

// Aucun paramètre
const aleatoire = () => Math.random();
```

Paramètres par défaut

```
function saluer(nom = "Inconnu", politesse = "Bonjour") {
  return `${politesse} ${nom}`;
}

saluer();              // "Bonjour Inconnu"
saluer("Alice");       // "Bonjour Alice"
saluer("Alice", "Salut"); // "Salut Alice"
```

Reste (rest parameter)

```
function somme(...nombres) {
  return nombres.reduce((acc, n) => acc + n, 0);
}

somme(1, 2, 3, 4);  // 10
```

Portée (*scope*)

```
const global = "Je suis partout";

function exemple() {
  const local = "Je suis seulement dans la fonction";
  console.log(global);  // OK
  console.log(local);   // OK
}

console.log(global);  // OK
// console.log(local);  // ❌ Erreur : non défini
```

---

9. Tableaux (arrays)

```
const fruits = ["pomme", "banane", "mangue"];

fruits[0]          // "pomme"
fruits.length      // 3
fruits[fruits.length - 1]  // "mangue"

// Ajouter / retirer
fruits.push("kiwi");      // À la fin
fruits.pop();              // Retire le dernier
fruits.unshift("fraise"); // Au début
fruits.shift();            // Retire le premier

// Recherche
fruits.includes("pomme")  // true
fruits.indexOf("banane")  // 1

// Modifier
fruits.splice(1, 1)             // Retire 1 élément à l'index 1
fruits.splice(1, 0, "orange")   // Insère "orange" à l'index 1

// Copier (sans modifier l'original)
const inverse = [...fruits].reverse();
const slice = fruits.slice(1, 3);
const concat = fruits.concat(["raisin"]);
```

Méthodes fonctionnelles (essentielles)

```
const nombres = [1, 2, 3, 4, 5];

// forEach : exécuter pour chaque élément
nombres.forEach(n => console.log(n));

// map : transformer chaque élément
const doubles = nombres.map(n => n * 2);
// [2, 4, 6, 8, 10]

// filter : garder seulement ceux qui passent un test
const pairs = nombres.filter(n => n % 2 === 0);
// [2, 4]

// reduce : agréger en une valeur unique
const somme = nombres.reduce((acc, n) => acc + n, 0);
// 15

// find : premier élément qui passe un test
const premierGrand = nombres.find(n => n > 3);
// 4

// some / every : booléens
nombres.some(n => n > 4)    // true
nombres.every(n => n > 0)   // true

// sort
[3, 1, 4, 1, 5].sort()                    // [1, 1, 3, 4, 5] (alphabétique !)
[3, 1, 4, 1, 5].sort((a, b) => a - b)     // Croissant
[3, 1, 4, 1, 5].sort((a, b) => b - a)     // Décroissant
```

10. Objets

```
const personne = {
  nom: "AGBAVON",
  prenom: "Kokou",
  age: 28,
  competences: ["HTML", "CSS", "JS"],
  saluer: function() {
    return `Bonjour, je suis ${this.prenom}`;
  }
};

// Accès
personne.nom              // "AGBAVON"
personne["nom"]           // "AGBAVON"
personne.saluer()         // "Bonjour, je suis Kokou"

// Modification
personne.age = 29;
personne.ville = "Lomé";  // Ajout

// Suppression
delete personne.ville;

// Parcours
Object.keys(personne)     // ["nom", "prenom", "age", …]
Object.values(personne)
Object.entries(personne)  // [["nom", "AGBAVON"], …]
```

Destructuration

```
const { nom, prenom } = personne;
console.log(nom, prenom);  // "AGBAVON Kokou"

// Avec renommage et valeur par défaut
const { nom: lastName, profession = "Étudiant" } = personne;

// Tableaux
const [premier, deuxieme, ...reste] = [1, 2, 3, 4, 5];
// premier=1, deuxieme=2, reste=[3,4,5]
```

Spread (`...`)

```
// Copier
const copie = { ...personne };

// Fusionner
const enrichi = { ...personne, ville: "Lomé", age: 30 };

// Tableaux
const tous = [...nombres1, ...nombres2];
```

---

11. Le DOM (Document Object Model)

Le **DOM** est la représentation **arborescente** d'une page HTML en mémoire. JavaScript peut le manipuler pour modifier la page **dynamiquement**.

```
document
  └── html
        ├── head
        │     └── title
        └── body
              ├── h1
              └── div
                    ├── p
                    └── img
```

Sélectionner des éléments

```
// Un seul élément
document.getElementById("monId");
document.querySelector(".maClasse");   // Premier match
document.querySelector("h1");
document.querySelector("nav > ul > li");

// Plusieurs
document.getElementsByClassName("item");   // HTMLCollection (vivante)
document.getElementsByTagName("p");        // HTMLCollection
document.querySelectorAll(".item");        // NodeList (statique, recommandée)
```

💡 Préférez `querySelector` et `querySelectorAll` : syntaxe CSS, plus puissants.

Modifier le contenu

```
const titre = document.querySelector("h1");

titre.textContent = "Nouveau titre";      // Texte brut
titre.innerHTML = "<em>Titre</em>";       // Avec HTML (attention XSS)
titre.innerText = "Idem mais respecte le style CSS";
```

Modifier les attributs

```
const lien = document.querySelector("a");

lien.href = "https://exemple.com";
lien.setAttribute("target", "_blank");
lien.getAttribute("href");
lien.removeAttribute("title");

// Dataset (attributs data-*)
// <div data-user-id="42">
const div = document.querySelector("div");
div.dataset.userId    // "42"
```

Modifier les classes

```
const carte = document.querySelector(".carte");

carte.classList.add("active");
carte.classList.remove("inactive");
carte.classList.toggle("ouvert");        // Ajoute si absent, retire si présent
carte.classList.contains("active");      // true/false
carte.classList.replace("ancien", "nouveau");
```

Modifier le style

```
carte.style.backgroundColor = "red";
carte.style.fontSize = "20px";
carte.style.display = "none";

// Préférable : modifier la classe et laisser CSS gérer
carte.classList.add("cachee");
```

Créer / insérer / supprimer

```
// Créer
const nouveau = document.createElement("li");
nouveau.textContent = "Nouvel élément";
nouveau.classList.add("item");

// Insérer
const liste = document.querySelector("ul");
liste.appendChild(nouveau);          // À la fin
liste.prepend(nouveau);              // Au début
liste.insertBefore(nouveau, autreElement);

// Méthodes modernes
parent.append(enfant1, enfant2);     // Plusieurs d'un coup
parent.prepend(enfant);
element.before(autre);
element.after(autre);

// Supprimer
element.remove();
parent.removeChild(enfant);

// Cloner
const copie = element.cloneNode(true);   // true = avec descendants
```

Naviguer dans le DOM

```
element.parentElement
element.children          // Enfants directs
element.firstElementChild
element.lastElementChild
element.nextElementSibling
element.previousElementSibling
element.closest(".carte") // Ancêtre le plus proche correspondant au sélecteur
```

---

12. Événements

Réagir aux interactions utilisateur (clic, survol, frappe…).

`addEventListener`

```
const bouton = document.querySelector("#monBouton");

bouton.addEventListener("click", function(event) {
  console.log("Bouton cliqué !");
  console.log(event);  // Objet Event
});

// Avec arrow function
bouton.addEventListener("click", (e) => {
  e.preventDefault();         // Empêcher comportement par défaut
  e.stopPropagation();        // Empêcher la propagation
});

// Retirer
bouton.removeEventListener("click", maFonction);
```

Événements courants

Catégorie	Événements
Souris	`click`, `dblclick`, `mousedown`, `mouseup`, `mouseenter`, `mouseleave`, `mousemove`, `contextmenu`
Clavier	`keydown`, `keyup`, `keypress`
Formulaire	`submit`, `change`, `input`, `focus`, `blur`, `reset`
Page	`load`, `DOMContentLoaded`, `resize`, `scroll`, `beforeunload`
Tactile	`touchstart`, `touchend`, `touchmove`
Glisser-déposer	`dragstart`, `dragend`, `drop`, `dragover`

Délégation d'événements

Plutôt que d'attacher un listener sur chaque enfant, on en met un seul sur le parent.

```
document.querySelector("#liste").addEventListener("click", (e) => {
  if (e.target.matches(".bouton-supprimer")) {
    e.target.closest("li").remove();
  }
});
```

---

13. Formulaires en JavaScript

Récupérer les valeurs

```
const formulaire = document.querySelector("#contact");
const champEmail = document.querySelector("#email");

console.log(champEmail.value);

// Avec FormData
formulaire.addEventListener("submit", (e) => {
  e.preventDefault();   // Empêche le rechargement

  const data = new FormData(formulaire);

  // Lire un champ
  console.log(data.get("email"));

  // Convertir en objet
  const objet = Object.fromEntries(data);
  console.log(objet);
});
```

Validation personnalisée

```
formulaire.addEventListener("submit", (e) => {
  const email = document.querySelector("#email").value;

  if (!email.includes("@")) {
    e.preventDefault();
    alert("E-mail invalide");
    return;
  }

  // Sinon le formulaire est soumis normalement
});
```

Événements de formulaire utiles

```
champ.addEventListener("input", (e) => {
  console.log("À chaque frappe :", e.target.value);
});

champ.addEventListener("change", (e) => {
  console.log("Quand le champ perd le focus si modifié");
});

champ.addEventListener("focus", () => { /* … */ });
champ.addEventListener("blur",  () => { /* … */ });
```

---

14. Stockage local (`localStorage`)

Stocker des données dans le navigateur (jusqu'à 5–10 Mo, persistant).

```
// Écrire
localStorage.setItem("theme", "sombre");
localStorage.setItem("user", JSON.stringify({ nom: "Alice" }));

// Lire
const theme = localStorage.getItem("theme");
const user = JSON.parse(localStorage.getItem("user"));

// Supprimer
localStorage.removeItem("theme");
localStorage.clear();   // Tout effacer
```

	`localStorage`	`sessionStorage`
Persistance	Permanent	Session (jusqu'à fermeture de l'onglet)
Capacité	~5–10 Mo	~5 Mo
Partage entre onglets	Oui (même origine)	Non

⚠️ Ne stockez **jamais** de mots de passe ou de jetons sensibles dans `localStorage`.

---

15. Asynchrone : Promises et async/await

JavaScript est mono-thread, mais peut faire des opérations en **arrière-plan** (requêtes réseau, timers…).

Callbacks (ancien style)

```
setTimeout(() => {
  console.log("Après 2 secondes");
}, 2000);
```

Promises

```
const promesse = new Promise((resolve, reject) => {
  // Opération asynchrone
  setTimeout(() => {
    const reussi = true;
    if (reussi) resolve("Données reçues");
    else reject("Erreur survenue");
  }, 1000);
});

promesse
  .then(donnees => console.log(donnees))
  .catch(erreur => console.error(erreur))
  .finally(() => console.log("Toujours exécuté"));
```

async / await (recommandé)

```
async function chargerDonnees() {
  try {
    const reponse = await fetch("https://api.exemple.com/users");
    const donnees = await reponse.json();
    console.log(donnees);
  } catch (erreur) {
    console.error("Erreur :", erreur);
  }
}

chargerDonnees();
```

`async` rend une fonction asynchrone (elle renvoie automatiquement une Promise).
`await` met en pause jusqu'à ce que la Promise soit résolue.

---

16. `fetch` — Appels HTTP

Récupérer des données depuis un serveur ou une API.

GET (lecture)

```
async function recupererArticles() {
  const reponse = await fetch("https://jsonplaceholder.typicode.com/posts");

  if (!reponse.ok) {
    throw new Error(`Erreur HTTP : ${reponse.status}`);
  }

  const articles = await reponse.json();
  return articles;
}

recupererArticles().then(articles => {
  articles.forEach(a => console.log(a.title));
});
```

POST (envoi)

```
async function creerArticle(data) {
  const reponse = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(data)
  });

  return await reponse.json();
}

creerArticle({ title: "Mon article", body: "Contenu", userId: 1 });
```

Exemple complet : afficher une liste depuis une API

```
async function afficherUtilisateurs() {
  const ul = document.querySelector("#liste-users");
  ul.innerHTML = "<li>Chargement…</li>";

  try {
    const reponse = await fetch("https://jsonplaceholder.typicode.com/users");
    const users = await reponse.json();

    ul.innerHTML = users
      .map(u => `<li>${u.name} — ${u.email}</li>`)
      .join("");
  } catch (e) {
    ul.innerHTML = `<li>Erreur : ${e.message}</li>`;
  }
}

afficherUtilisateurs();
```

---

17. JSON

JSON (*JavaScript Object Notation*) est le format d'échange de données du Web.

```
{
  "nom": "AGBAVON",
  "prenom": "Kokou",
  "age": 28,
  "competences": ["HTML", "CSS", "JS"],
  "adresse": {
    "ville": "Lomé",
    "pays": "Togo"
  }
}
```

Fonction	Rôle
`JSON.stringify(obj)`	Objet JS → chaîne JSON
`JSON.parse(str)`	Chaîne JSON → objet JS

Différences avec un objet JS :

- Les clés doivent être entre **guillemets doubles**.
- Pas de virgule finale.

- Pas de fonctions, ni `undefined`.

---

18. Gestion des erreurs

```
try {
  // Code susceptible d'échouer
  const data = JSON.parse("invalid");
} catch (erreur) {
  console.error("Erreur attrapée :", erreur.message);
} finally {
  console.log("Toujours exécuté");
}

// Lancer une erreur volontairement
function diviser(a, b) {
  if (b === 0) throw new Error("Division par zéro !");
  return a / b;
}
```

---

19. Modules ES6

Organiser le code en plusieurs fichiers.

`utils.js` :

```
export function additionner(a, b) {
  return a + b;
}

export const PI = 3.14159;

export default function principal() {
  console.log("Export par défaut");
}
```

`main.js` :

```
import principal, { additionner, PI } from "./utils.js";

principal();
console.log(additionner(2, 3));
console.log(PI);
```

`index.html` :

```
<script type="module" src="main.js"></script>
```

---

20. Bonnes pratiques

✅ À faire

- Utiliser `const` par défaut, `let` si réassignation.
- Utiliser `===` au lieu de `==`.
- Préférer les fonctions fléchées pour les callbacks.
- Utiliser `defer` pour les scripts dans le `<head>`.
- Délégation d'événements pour les listes longues.
- Toujours gérer les erreurs (`try / catch`).
- Modulariser le code (ES modules).
- Utiliser un linter (ESLint).

❌ À éviter

- `var`.
- `==` et `!=`.
- `document.write()`.
- `eval()`.
- `alert()` / `confirm()` / `prompt()` en production.
- `innerHTML` avec des données utilisateur (risque XSS).
- Variables globales.

---

21. Quiz d'auto-évaluation

1. Quelle est la différence entre `let` et `const` ?
2. Pourquoi préférer `===` à `==` ?
3. Que renvoie `typeof null` ? Est-ce normal ?
4. Quelle méthode de tableau renvoie un nouveau tableau transformé ?
5. Comment sélectionner tous les `<p>` d'une page ?
6. Quelle est la différence entre `textContent` et `innerHTML` ?
7. À quoi sert `e.preventDefault()` dans un formulaire ?
8. Différence entre `localStorage` et `sessionStorage` ?
9. Que fait `await` ?
10. À quoi sert `JSON.stringify` ?

> **Réponses** : 1. `const` non réassignable · 2. `===` ne fait pas de conversion de type · 3. `"object"` — bug historique conservé · 4. `map()` · 5. `document.querySelectorAll("p")` · 6. `textContent` = texte brut sans HTML · 7. Empêcher le rechargement de page · 8. `localStorage` persiste / `sessionStorage` jusqu'à fermeture de l'onglet · 9. Attend la résolution d'une Promise · 10. Convertir un objet JS en chaîne JSON.

Ressources

- 📚 MDN — Apprendre JavaScript
- 📚 JavaScript.info — tutoriel moderne en français
- 📚 Eloquent JavaScript — livre gratuit (anglais)
- 📚 You Don't Know JS — série de référence
- 📚 JSONPlaceholder — API de test
- 🎓 Codecademy JavaScript

➡️ **Module suivant** : 05 — WordPress